

Introduction

Deep Learning Models like FourCastNet have revolutionized the field by providing forecasts that are orders of magnitude faster while maintaining comparable accuracy. However, as these models move toward high-resolution global grids (), their computational footprint—specifically VRAM usage and inference latency; becomes a bottleneck for deployment on edge devices or workstations with limited resources.

Problem Statement

The baseline FourCastNet model, while faster than traditional, still presents significant deployment challenges:

- **High Latency:** A 12-layer backbone takes approximately 142 ms for a single-step inference, which accumulates during long-term.
- **Memory Constraints:**
- **Storage Overhead:** Standard FP32 checkpoint require ~300 MB of disk space.

About the Model: AFNONet Architecture

FourCastNet is built on the Adaptive Fourier Neural Operator (AFNO). Unlike standard vision Transformers that use spatial self-attention (which scales quadratically with resolution), AFNO operates in the frequency domain.

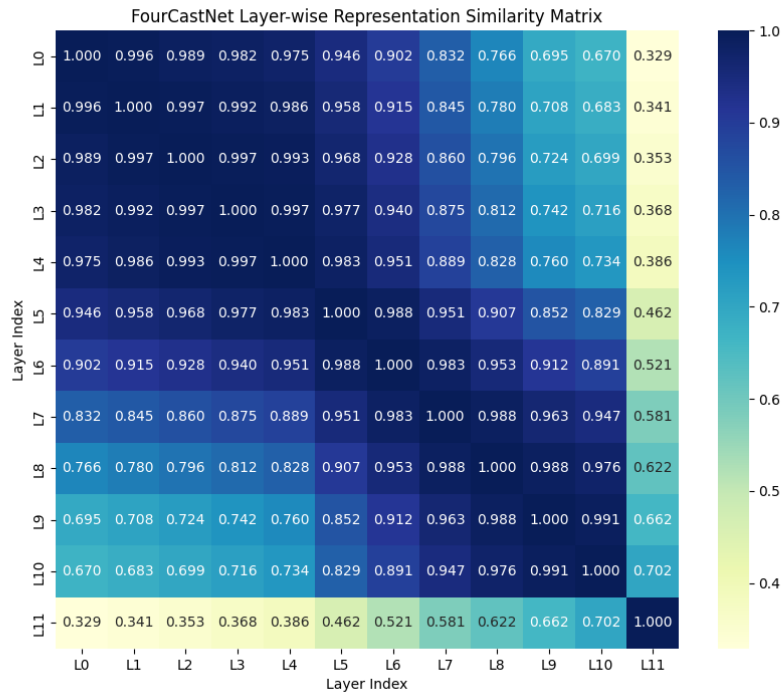
- **Global Token Mixing:** It uses 2D Fast Fourier Transforms (FFT) to capture global dependencies across the entire atmospheric grid.
- **Spectral Filtering:** Learnable complex weights are applied to the Fourier coefficients using a block diagonal matrix approach to maintain efficiency.
- **Backbone:** The baseline model consists of 12 identical AFNO blocks, a patch embedding layer, and a prediction head.

Dataset (ERA5)

Compression Methodologies and Results

1. Architectural Pruning (Depth Reduction)

Since 97.3% of the latency is covered by the Transformer blocks, we tested depth reduction. I used cosine similarity matrix between each representation of AFNO block for dropping each block. Layers having greater similarity will be dropped first.



According to this, the order of dropping AFNO blocks would be [3, 4, 2, 1, 10, 9] of of 12 blocks (0 to 11).

We are using application specific compression of deep learning models for this pruning.

(<https://dl.acm.org/doi/epdf/10.1145/3703323.3703339>)

- Method: The model was pruned from 12 to 6 layers.
- Result:
 - Latency Dropped from 142 ms to 73 ms (~1.95x speedup)
 - Parameters: Reduced from 74.6M to 44.5M (~40% reduction)

With full backbone model gives:

Avg Latency: 142.19 ms

Avg Z500 RMSE: 36.97 m²/s².

Avg T850 RMSE: 0.52 K

Avg Z500 ACC: 0.9994

Table 1: RMSE (m^2/s^2) Evaluation Across Pruned Configurations before and after fine-tuning

# pruned layers	1 year (2015)						2 years (2014-2015)						
	base	1	2	3	4	5	6	1	2	3	4	5	6
1	260.32	39.38	39.1	39.25	39.05	39.18	39.3	38.43	38.2	38.35	38.1	38.3	38.18
2	494.56	41.46	41.4	41.18	41.35	41.3	41.22	40.81	40.55	40.7	39.6	39.85	39.65
3	557.56	43.54	43.7	43.11	43.65	43.42	43.14	43.19	42.9	43.05	41.1	41.4	41.12
4	717.63	45.62	46	45.04	45.95	45.54	45.06	45.57	45.25	45.4	42.6	42.95	42.59
5	739.93	47.7	48.3	46.97	48.25	47.66	46.98	47.95	47.6	47.75	44.1	44.5	44.06
6	818.82	49.78	50.6	48.9	50.55	49.78	48.9	50.33	49.95	50.1	45.6	46.05	45.53

# of pruned layers	Fine-Tuning on 2014-2015 dataset for # of Epochs							Latency/step (ms)
	base	1	2	3	4	5	6	
0	36.97							
1	191.10	37.0	36.53	36.72	35.64	36.22	35.00	130 (1.0x)
2	365.04	41.49	39.71	38.79	38.78	37.98	37.44	119 (1.09x)
3	569.02	48.54	44.21	42.80	41.88	41.0	40.63	107 (1.19x)
4	817.76	64.61	53.87	48.74	46.88	45.06	44.27	96 (1.32x)
5	1061.37	70.49	58.74	54.33	50.57	48.76	47.17	84 (1.48x)
6	1297.22	84.56	69.99	60.07	56.63	54.68	52.51	731.95x)

I tried to run all inference in isolated environment to calculate how latency and parameter counts changes as we perform layer pruning as well as input compression.

	Total	Transformer	Embedding/Head/Pos	Latency (ms)	speed-up
base (12 Layer)	74,691,840	60,281,856	14,409,984	142.21	1.0x
pruned 1 layer	69,668,352	55,258,368	14,409,984	130.68	1.09x
pruned 2 layer	64,644,864	50,234,880	14,409,984	119.23	1.19x
pruned 3 layer	59,621,376	45,211,392	14,409,984	107.49	1.32x
pruned 4 layer	54,597,888	40,187,904	14,409,984	96.04	1.48x
pruned 5 layer	49,574,400	35,164,416	14,409,984	84.39	1.69x
pruned 6 layer	44,550,912	30,140,928	14,409,984	73.03	1.95x

Observations:

- Transformer part covers **80.7%** of the model and rest of the components hold **19.3%**.
- Each Transformer Block is of Approx **5.02 million** params.
- Reducing the model to 6 layers (50% depth reduction) yields speedup in latency (~1.95x), bringing inference time down to **73.03 ms**.
- The "Rest" of the model (Embedding, Head, and Positional encodings) remains constant at **14.4 million** parameters.

2. Input Channel Pruning (Variable Reduction)

I performed a "Leave-One-Out" sensitivity analysis on the 20 input variables to identify candidates for redundancy reduction.

The patch_embed layer is a 2D Convolution that projects these 20 channels into the embed_dim (768).

We analysed the 20 input variables (e.g. Temperature, U/V Winds, Geopotential heights) to identify redundancy.

- Method: A "Leave-One-Out" sensitivity analysis was performed by masking individual channels and measuring the RMSE impact on Z_{500} .
- Findings: Variables in the Moisture Group (tcwv, r500) showed the lowest sensitivity, while dynamical core variables (z500, v500) were critical.
- Result: Reduced the input from 20 to 14 channels, optimizing the I/O pipeline and initial embedding parameters by 30%.

These are 20 channels FourCastNet have:

Category	Index	Variable Names	Physical Description
Surface (Near-Ground)	0	u10	10m U-velocity (Eastward wind)
	1	v10	10m V-velocity (Northward wind)
	2	t2m	2-meter Temperature
	3	sp	Surface Pressure
	4	mslp	Mean Sea Level Pressure
Upper-Air Dynamics	5	t850	Temperature at 850 hPa
	6,7	u1000, v1000	Winds at 1000 hPa
	8	z1000	Geopotential at 1000 hPa
	9,10	u850, v850	Winds at 850 hPa
	11	z850	Geopotential at 850 hPa
	12,13	u500, v500	Winds at 500 hPa
	14	z500	Geopotential at 500 hPa
	15	t500	Temperature at 500 hPa
	16	z50	Geopotential at 50 hPa (Stratosphere)
Moisture & Water	17	r500	Relative Humidity at 500 hPa
	18	r850	Relative Humidity at 850 hPa
	19	tcwv	Total Column Water Vapor

Importance Category	Variable	RMSE (m2/s2)	ACC
I. Non-Essential (Lowest Sensitivity)	r500	39.21	0.9993
	r850	44.00	0.9991
	t500	47.02	0.9989
	v10	53.46	0.9987
	v1000	56.28	0.9985
II. Low Impact	v850	68.38	0.9981
	u1000	101.33	0.9952
	u850	101.96	0.9952
	u10	122.38	0.9925
	Sp	123.32	0.9935
III. Moderate Impact	z50	124.38	0.9940
	u500	134.02	0.9928
	Tcwv	215.99	0.9785
	Mslp	328.39	0.9484
	t2m	742.57	0.7583
IV. Critical	t850	794.60	0.7691
	z1000	2330.40	0.2560
	z850	5662.06	0.1853
V. Mandatory (Highest Sensitivity)	z500	8980.41	-0.1836
	v500	10375.14	-0.0402

20 channels:- 983,808 parameters

14 channels:- 688,896 parameters

We save around **294,912 params**, which is **0.4%** of total 74,691,840 parameters. It does not significantly shrink the model file size and No impact on latency also.

3. Precision Quantization (FP32 to FP16)

We targeted the storage and memory footprint through weight precision reduction.

- Method: Quantization was used to convert weights to Half-Precision (FP16)
- **Storage & Memory Efficiency:** Quantization effectively doubles your hardware capacity by cutting the memory and storage requirements exactly in half.
- **Latency vs. Implementation:** The speed improvement (latency drop) is only realized if the entire computational graph—including the input tensors—is cast to `.half()`. If loaded as FP16 but executed in `.float()`, the latency remains at the baseline (142.21 ms).
- **Result:**
 - Size on Disk: Reduced from 284.93 MB to 140 MB.
 - Accuracy: Maintained a high Z₅₀₀ ACC of 0.9994.

Inference in FP32:

Avg Latency: 142.18 ms

Avg Z500 RMSE: 37.07 m²/s².

Avg T850 RMSE: 0.52 K

Avg Z500 ACC: 0.9994

Inference in FP16:

Avg Latency: 46.72 ms

Avg Z500 RMSE: 37.28 m²/s².

Avg T850 RMSE: 0.52 K

Avg Z500 ACC: 0.9993

4. Combine Architectural pruning and FP16 Quantization

I have first implemented layer pruning and the quantization in to FP16.

# of pruned layers	Fine-tuned FP32		Fine-Tuned FP16	
	Avg Z500 RMSE (m ² /s ²)	Latency (ms)	Avg Z500 RMSE (m ² /s ²)	Latency (ms)
1	35.00	130 ms	35.33	42.98
2	37.44	119 ms	37.70	39.19
3	40.63	107 ms	40.71	35.44
4	44.27	96 ms	44.40	31.57
5	47.17	84 ms	47.27	27.86
6	52.51	73 ms	52.46	24.07